COMP108
Data Structures and Algorithms

Divide-and-Conquer Algorithms (Part III)

Professor Prudence Wong

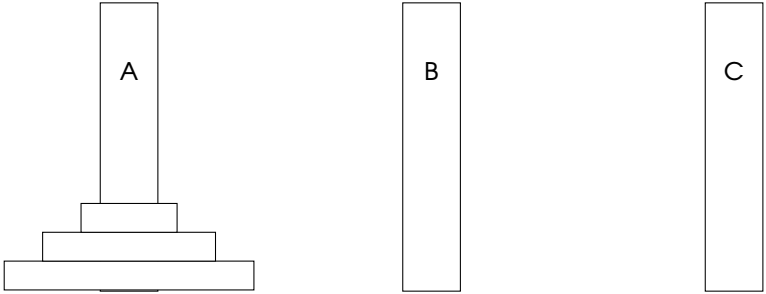
pwong@liverpool.ac.uk

2025-26

Tower of Hanoi

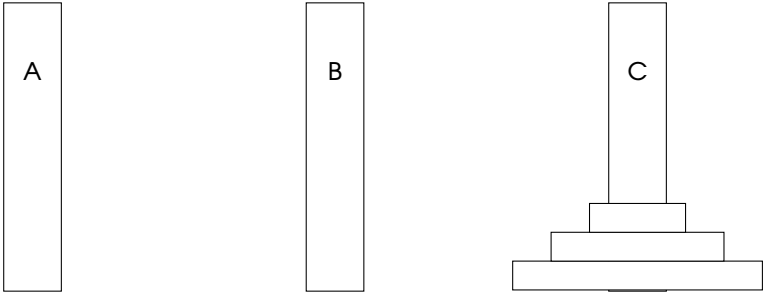
Tower of Hanoi

There are three pegs and some discs of different sizes are on Peg A.



Tower of Hanoi

Want to move the discs to Peg C.



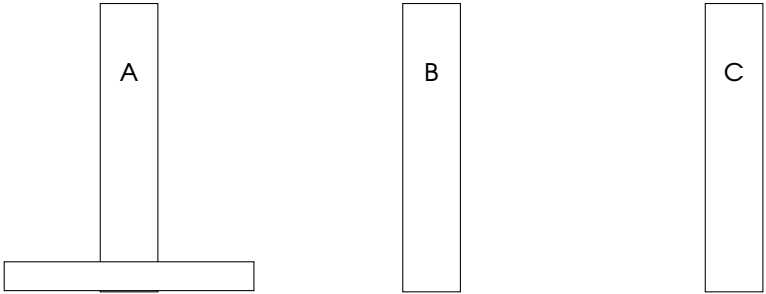
Tower of Hanoi - Rules

- Only 1 disk can be moved at a time
- A disc cannot be placed on top of other discs that are smaller than it

- **Target: Use the smallest number of moves**

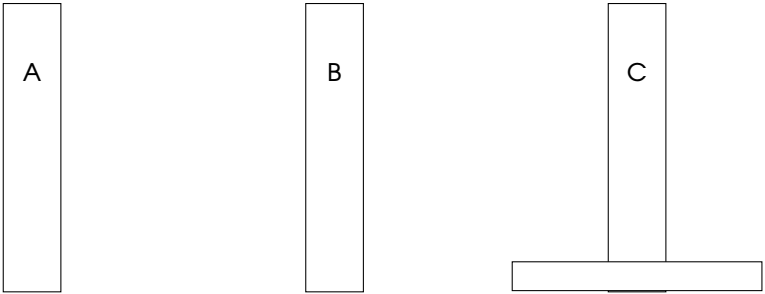
Tower of Hanoi - One Disc (Disc-L)

Easy! Need one move only.



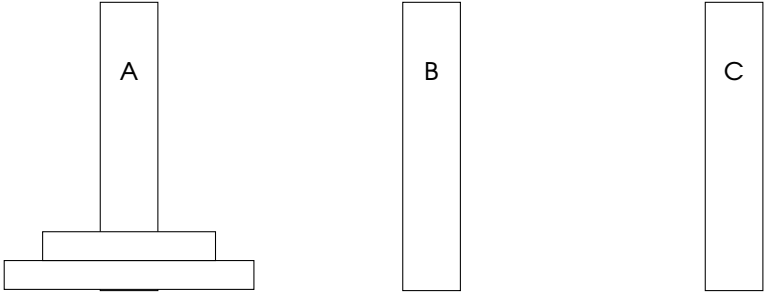
Tower of Hanoi - One Disc (Disc-L)

Easy! Need one move only.



Tower of Hanoi - Two Discs (Disc-L / Disc-M)

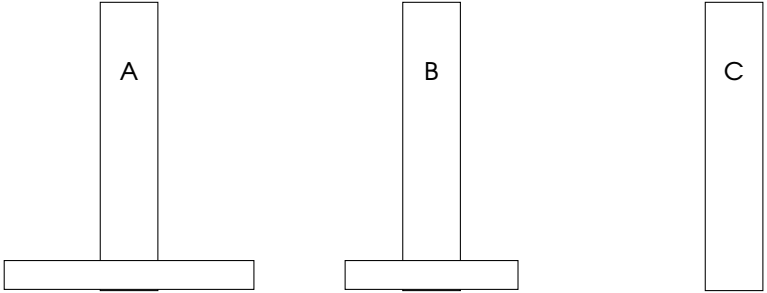
We need to move Disc-L to C, How?



Tower of Hanoi - Two Discs (Disc-L / Disc-M)

We need to move Disc-L to C, How?

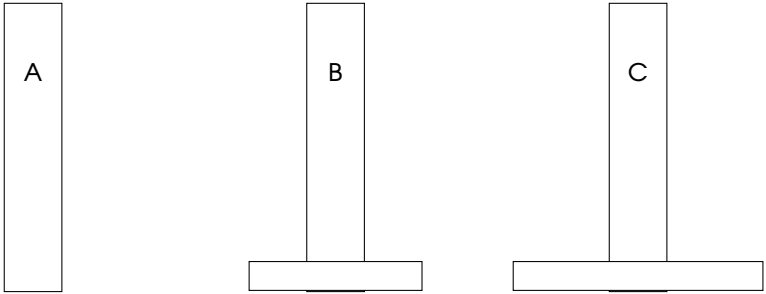
by moving Disc-M to B first,



Tower of Hanoi - Two Discs (Disc-L / Disc-M)

We need to move Disc-L to C, How?

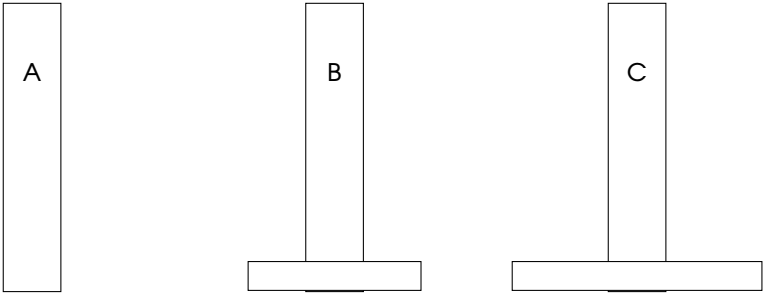
by moving Disc-M to B first, then Disc-L to C



Tower of Hanoi - Two Discs (Disc-L / Disc-M)

We need to move Disc-L to C, How?
by moving Disc-M to B first, then Disc-L to C

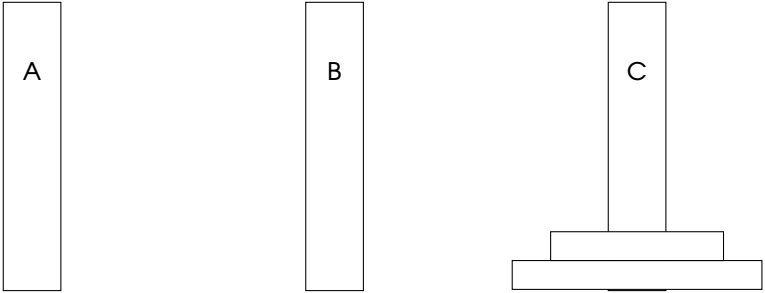
Next?



Tower of Hanoi - Two Discs (Disc-L / Disc-M)

We need to move Disc-L to C, How?
by moving Disc-M to B first, then Disc-L to C

Next? Move Disc-M to C

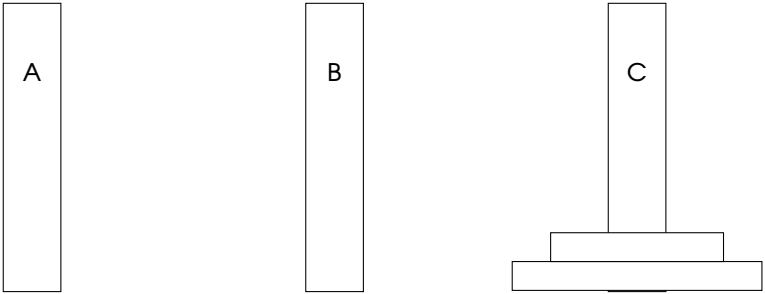


Tower of Hanoi - Two Discs (Disc-L / Disc-M)

We need to move Disc-L to C, How?
by moving Disc-M to B first, then Disc-L to C

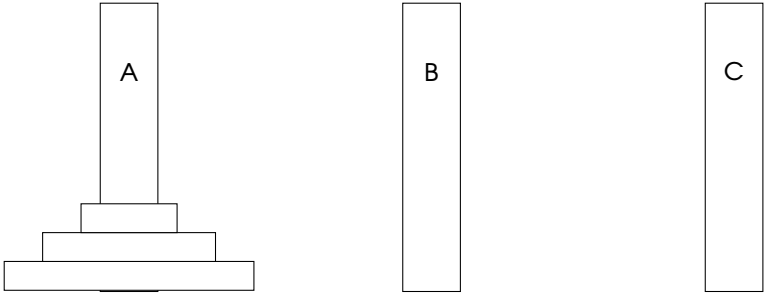
Next? Move Disc-M to C

3 moves needed



Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S)

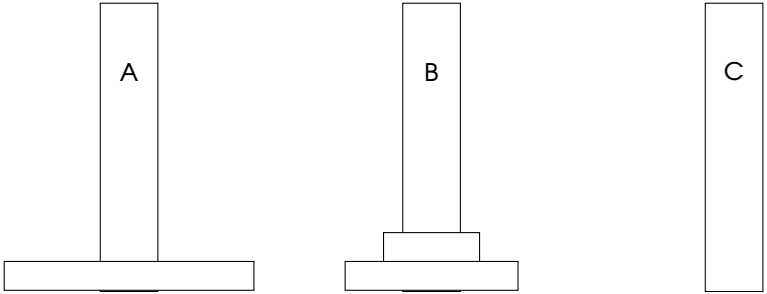
We need to move Disc-L to C, How?



Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S)

We need to move Disc-L to C, How?

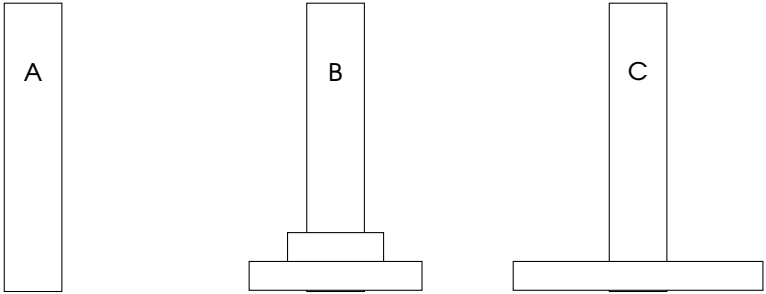
- moving Disc-M & Disc-S (two discs) from A to B **recursively**



Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S)

We need to move Disc-L to C, How?

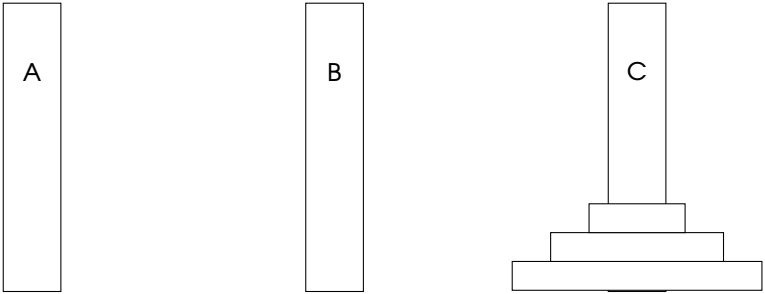
- moving Disc-M & Disc-S (two discs) from A to B **recursively**
- moving Disc-L from A to C



Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S)

We need to move Disc-L to C, How?

- moving Disc-M & Disc-S (two discs) from A to B **recursively**
- moving Disc-L from A to C
- moving Disc-M & Disc-S (two discs) from B to C **recursively**

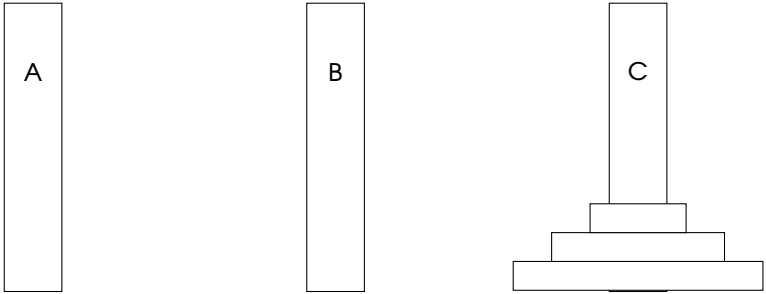


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S)

We need to move Disc-L to C, How?

- moving Disc-M & Disc-S (two discs) from A to B **recursively**
- moving Disc-L from A to C
- moving Disc-M & Disc-S (two discs) from B to C **recursively**

Moves needed? 3 + 1 + 3 = 7

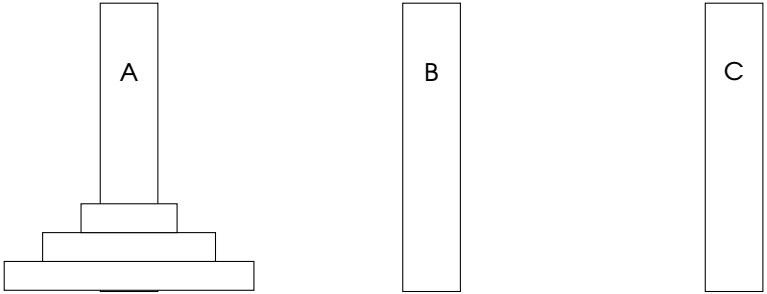


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** -

Moves needed? 3 + 1 + 3 = 7

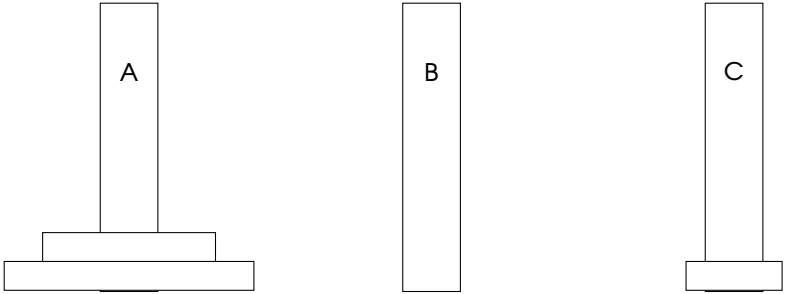


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C

Moves needed? 3 + 1 + 3 = 7

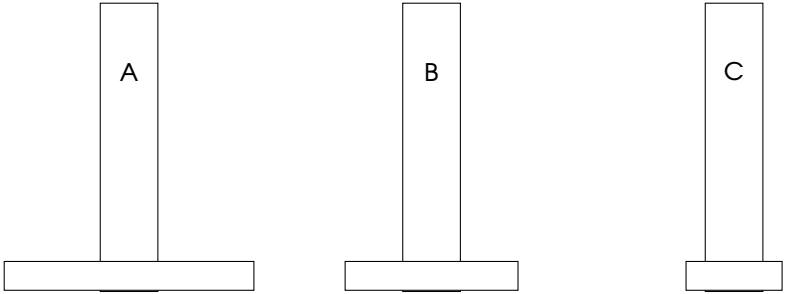


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B

Moves needed? 3 + 1 + 3 = 7

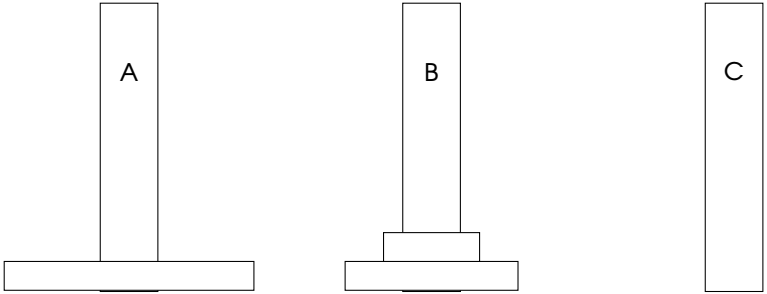


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B / S: C to B

Moves needed? 3 + 1 + 3 = 7

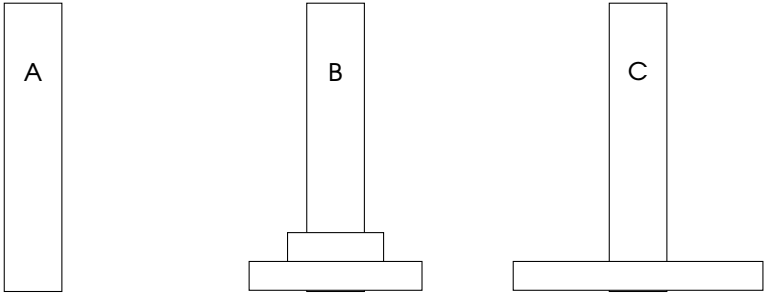


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B / S: C to B
- moving L from A to C

Moves needed? 3 + 1 + 3 = 7

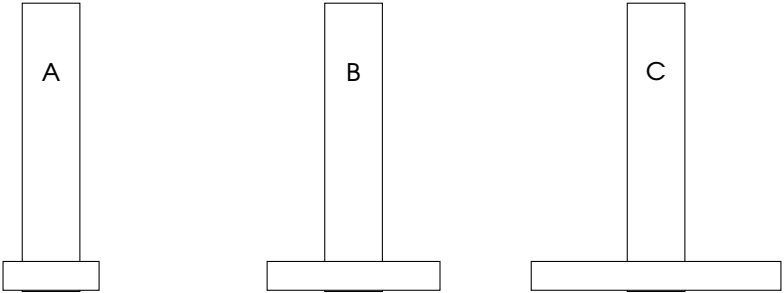


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B / S: C to B
- moving L from A to C
- M & S from B to C **recursively** - S: B to A

Moves needed? 3 + 1 + 3 = 7

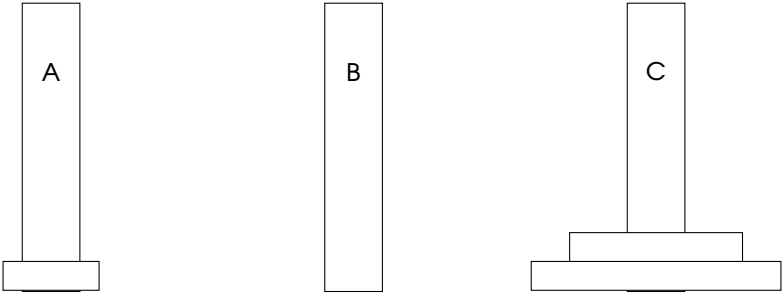


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B / S: C to B
- moving L from A to C
- M & S from B to C **recursively** - S: B to A / M: B to C

Moves needed? 3 + 1 + 3 = 7

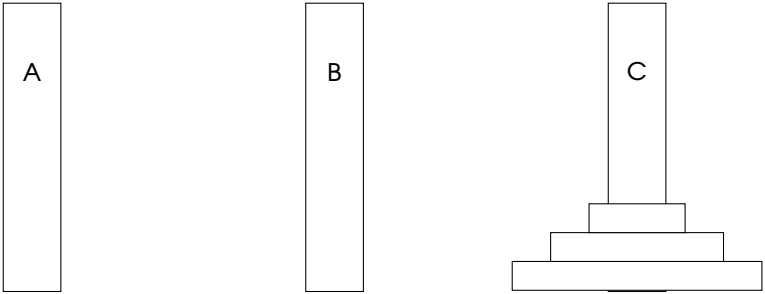


Tower of Hanoi - Three Discs (Disc-L / Disc-M / Disc-S) - Details

We need to move Disc-L to C, How?

- M& S from A to B **recursively** - S: A to C / M: A to B / S: C to B
- moving L from A to C
- M & S from B to C **recursively** - S: B to A / M: B to C / S: A to C

Moves needed? 3 + 1 + 3 = 7



Tower of Hanoi - More Discs

What if there are more discs?

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31
- ⋮

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31
- ⋮
- n discs: move n-1 from A to B recursively, move largest from A to C, move n-1 from B to C recursively

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15 $= \mathbf{2^4 - 1}$
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31
- $\vdots$
- n discs: move n-1 from A to B recursively, move largest from A to C, move n-1 from B to C recursively

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15 $= \mathbf{2^4 - 1}$
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31 $= \mathbf{2^5 - 1}$
- $\vdots$
- n discs: move n-1 from A to B recursively, move largest from A to C, move n-1 from B to C recursively

Tower of Hanoi - More Discs

What if there are more discs?

$2^n - 1$

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15 $= 2^4 - 1$
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31 $= 2^5 - 1$
- ⋮
- n discs: move n-1 from A to B recursively, move largest from A to C, move n-1 from B to C recursively
  # moves: $= 2^n - 1$

-1

$(2^{n-1} - 1) + 1 + (2^{n-1} - 1)$

$2^n = 2 \times 2^{n-1} - 1 + 1 - 1$

Tower of Hanoi - More Discs

What if there are more discs?

- 4 discs: move 3 from A to B recursively, move largest from A to C, move 3 from B to C recursively
  # moves: 7 + 1 + 7 = 15 $= 2^4 - 1$
- 5 discs: move 4 from A to B recursively, move largest from A to C, move 4 from B to C recursively
  # moves: 15 + 1 + 15 = 31 $= 2^5 - 1$
- $\vdots$
- n discs: move n-1 from A to B recursively, move largest from A to C, move n-1 from B to C recursively
  # moves: $= 2^n - 1$

exponential!

Tower of Hanoi - Pseudo Code

ToH(numDisc, source, destination, spare)

ToH(n, A, C, B)

Tower of Hanoi - Pseudo Code

```
ToH(numDisc, source, destination, spare)

    if (numDisc > 1) then
```

Tower of Hanoi - Pseudo Code

ToH(numDisc, source, destination, spare)

if (numDisc > 1) then

A B C

ToH(numDisc-1, source, spare, destination)

(n, A, C, B)

↓

(n-1, A, B, C)

move largest

(n-1, B, C, A)

Tower of Hanoi - Pseudo Code

```
ToH(numDisc, source, destination, spare)
    if (numDisc > 1) then
        ToH(numDisc-1, source, spare, destination)
    Move a disc from source to destination
```

Tower of Hanoi - Pseudo Code

```
ToH(numDisc, source, destination, spare)
    if (numDisc > 1) then
        ToH(numDisc-1, source, spare, destination)
    Move a disc from source to destination
    if (numDisc > 1) then
```

Tower of Hanoi - Pseudo Code

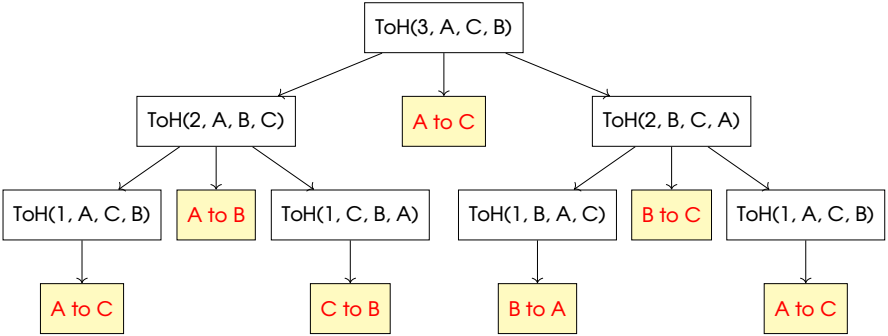
```
ToH(numDisc, source, destination, spare)
    if (numDisc > 1) then
        ToH(numDisc-1, source, spare, destination)
    Move a disc from source to destination
    if (numDisc > 1) then
        ToH(numDisc-1, spare, destination, source)
```

Tower of Hanoi - Pseudo Code

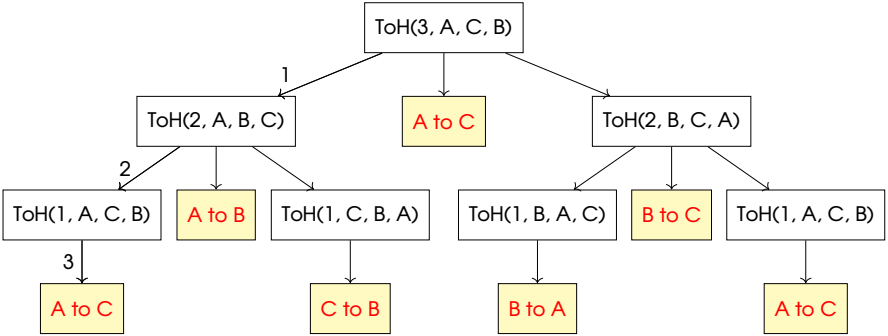
```
ToH(numDisc, source, destination, spare)
    if (numDisc > 1) then
        ToH(numDisc-1, source, spare, destination)
    Move a disc from source to destination
    if (numDisc > 1) then
        ToH(numDisc-1, spare, destination, source)
```

Invoke by ToH(3, A, C, B)

Execution of ToH(3, A, C, B)

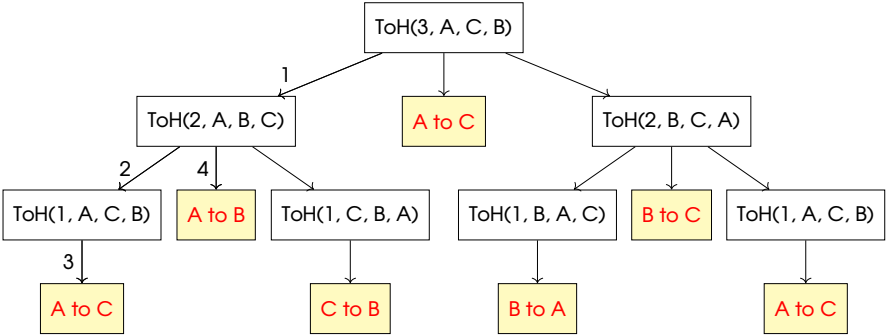


Execution of ToH(3, A, C, B)



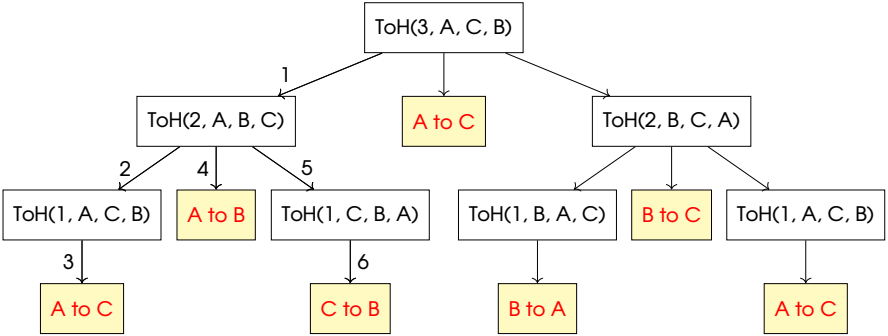
A to C;

Execution of ToH(3, A, C, B)



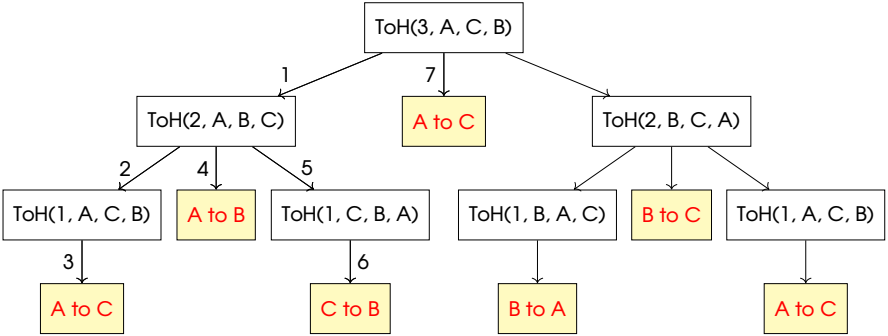
A to C; A to B;

Execution of ToH(3, A, C, B)



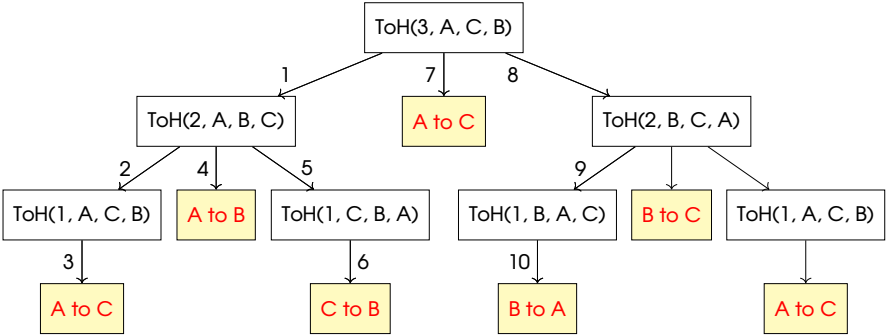
A to C; A to B; C to B;

Execution of ToH(3, A, C, B)



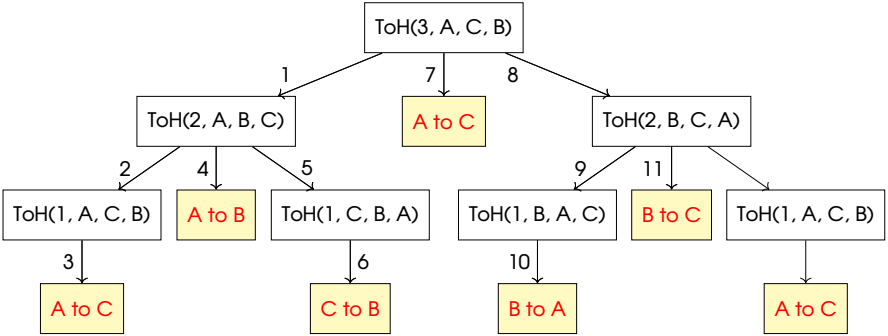
A to C; A to B; C to B;
A to C;

Execution of ToH(3, A, C, B)



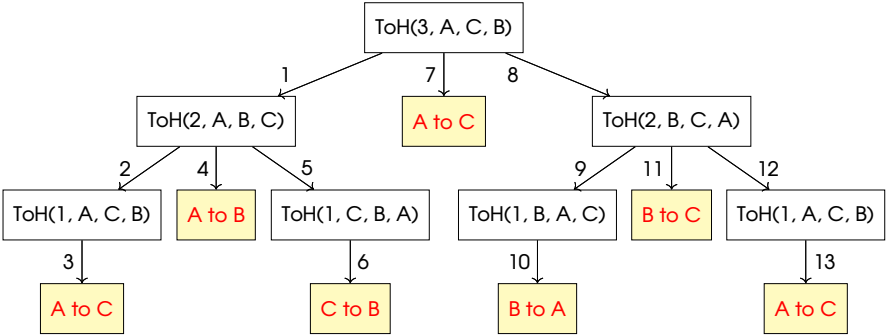
A to C; A to B; C to B;
A to C;
B to A;

Execution of ToH(3, A, C, B)



A to C; A to B; C to B;
A to C;
B to A; B to C;

Execution of ToH(3, A, C, B)



A to C; A to B; C to B;
A to C;
B to A; B to C; A to C;

Fibonacci Numbers

Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.

end of month-0

Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.

end of month-0

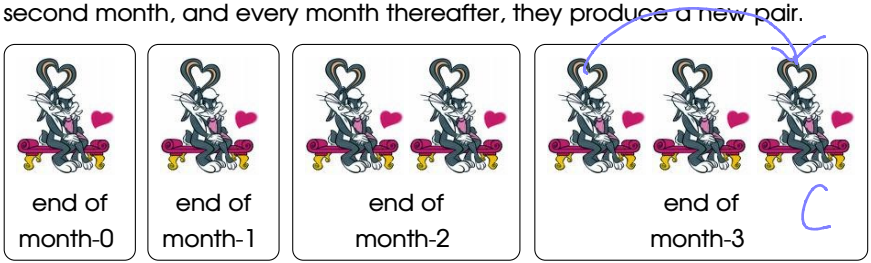
end of month-1

Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.

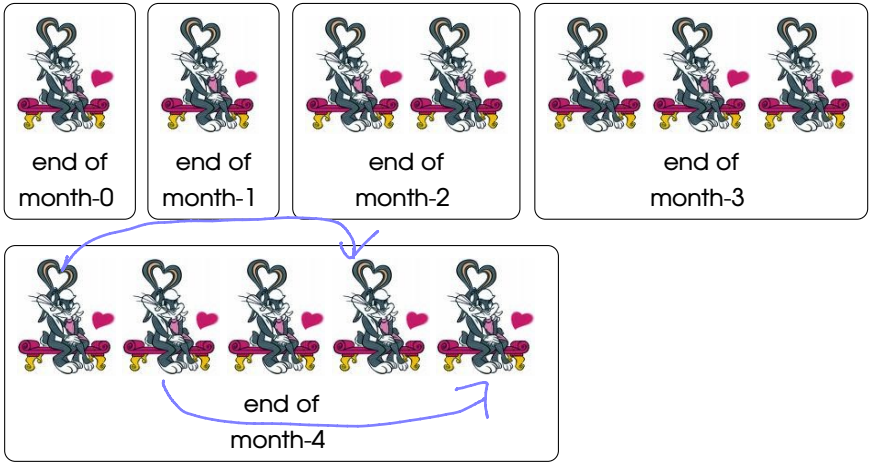
Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.



Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.



Fibonacci Rabbits

A pair of rabbits, one month old, is too young to reproduce. Suppose that in their second month, and every month thereafter, they produce a new pair.

end of month-0

end of month-1

end of month-2

end of month-3

end of month-4

How many at end of month-5, 6, 7 and so on?

8, 13, 21, 34

Fibonacci Numbers in Nature - Petals on flowers

1 petal:
white calla lily

2 petals:
euphorbia

3 petals:
trillium

5 petals:
columbine

8 petals:
bloodroot

13 petals:
black-eyed susan

21 petals:
shasta daisy

34 petals:
field daisy

Fibonacci Numbers

Fibonacci number $F(n)$

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Fibonacci Numbers

Fibonacci number $F(n)$

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	1	1	2	3	5	8	13	21	34	55	89

Fibonacci Numbers SampleFib.java

Fibonacci number $F(n)$

recurrence

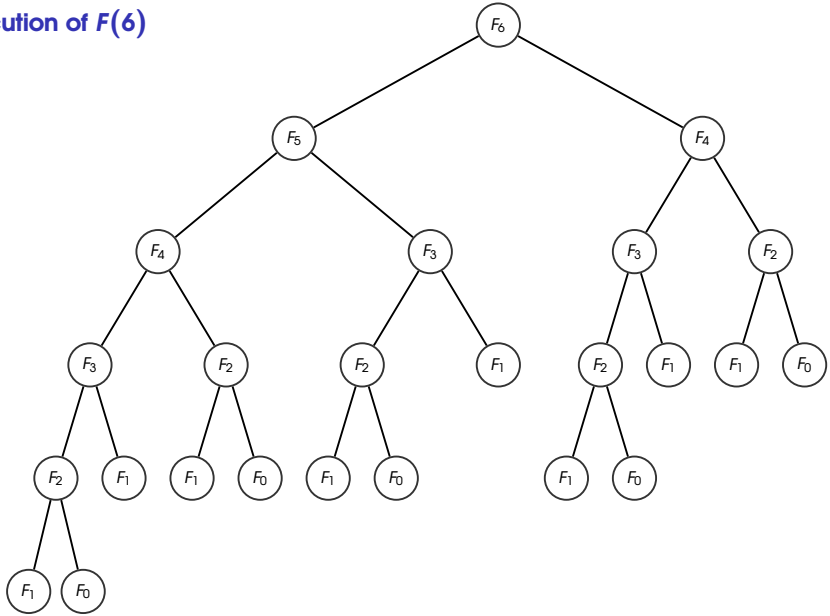
$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	1	1	2	3	5	8	13	21	34	55	89

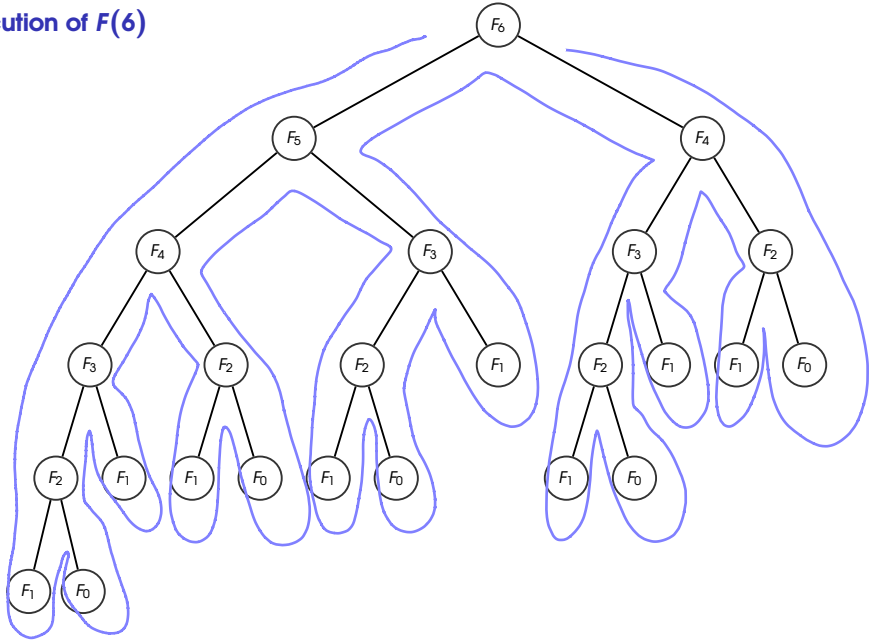
Pseudo code for the recursive algorithm:

```
Algorithm F(n)
    if n == 0 OR n == 1 then
        return 1
    else
    return F(n-1) + F(n-2)
```

Execution of $F(6)$



Execution of $F(6)$



Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- $T(n-1)$: time to calculate $F(n-1)$,

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- $T(n-1)$: time to calculate $F(n-1)$,
- $T(n-2)$: time to calculate $F(n-2)$,

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- $T(n-1)$: time to calculate $F(n-1)$,
- $T(n-2)$: time to calculate $F(n-2)$,
- plus 1: time to add $F(n-1)$ and $F(n-2)$

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Time Complexity

To analyze the time complexity of calculating Fibonacci numbers, we use a mathematical tool called **recurrence**.

Let $T(n)$ denote the time to calculate Fibonacci number $F(n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- $T(n-1)$: time to calculate $F(n-1)$,
- $T(n-2)$: time to calculate $F(n-2)$,
- plus 1: time to add $F(n-1)$ and $F(n-2)$
- When n is 0 or 1, there is no need to divide and the only time needed is to return the number 1.

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

To solve a recurrence is to derive asymptotic bounds on the solution

Recurrence for Merge Sort

Let $T(n)$ denote the time complexity of running merge sort on n numbers.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

- $T(\frac{n}{2})$: time to recursively sort one half
- 2 times : there are two halves
- plus n: to merge two sorted halves
- When n is 1, there is no need to divide and the only time needed is to return the number itself

Recurrence for Finding Sum/Max/Min

Let $T(n)$ denote the time complexity of finding sum/max/min on n numbers.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

- $T(\frac{n}{2})$: time to recursively search one half
- 2 times : there are two halves
- plus 1: only one addition or one comparison to combine the two answers
- When n is 1, there is no need to divide and the only time needed is to return the number itself

Solving Recurrence

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$T(n) = T(n-1) + T(n-2) + 1$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \end{aligned}$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\ &> 2 \times T(n-2) \end{aligned}$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4))
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4)
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6))
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6)
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8))
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4)
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots \\
&> 2^{\frac{n}{2}} \times T(0)
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots \\
&> 2^{\frac{n}{2}} \times T(0) \\
&= 2^{\frac{n}{2}}
\end{aligned}
$$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots \\
&> 2^{\frac{n}{2}} \times T(0) \\
&= 2^{\frac{n}{2}}
\end{aligned}
$$

Suppose n is odd. $\quad T(n) > 2^{\frac{n-1}{2}} \times T(1)$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots \\
&> 2^{\frac{n}{2}} \times T(0) \\
&= 2^{\frac{n}{2}}
\end{aligned}
$$

Suppose n is odd. $\quad T(n) > 2^{\frac{n-1}{2}} \times T(1) = 2^{\frac{n-1}{2}}$

Solving recurrence for calculating Fibonacci numbers

Suppose n is even.

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + 1 \\
&= (T(n-2) + T(n-3) + 1) + T(n-2) + 1 \\
&> 2 \times T(n-2) \\
&> 2 \times (2 \times T(n-4)) = 2^2 \times T(n-4) \\
&> 2^2 \times (2 \times T(n-6)) = 2^3 \times T(n-6) \\
&> 2^3 \times (2 \times T(n-8)) = 2^4 \times T(n-2 \times 4) \\
&\vdots \\
&> 2^k \times T(n-2k) \\
&\vdots \\
&> 2^{\frac{n}{2}} \times T(0) \\
&= 2^{\frac{n}{2}}
\end{aligned}
$$

Suppose n is odd. $\quad T(n) > 2^{\frac{n-1}{2}} \times T(1) = 2^{\frac{n-1}{2}}$

exponential!

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$T(n) \quad = \quad 2 \times T\left(\frac{n}{2}\right) + n$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n \end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \\ &= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}) + 2n \end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \\ &= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}) + 2n = 2^3 \times T\left(\frac{n}{2^3}\right) + 3n \\ &\vdots \end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \\ &= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}) + 2n = 2^3 \times T\left(\frac{n}{2^3}\right) + 3n \\ &\vdots \\ &= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (\log n) \times n \end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned}
T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\
&= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \\
&= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}) + 2n = 2^3 \times T\left(\frac{n}{2^3}\right) + 3n \\
&\vdots \\
&= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (\log n) \times n \\
&= 2^{\log n} \times T(1) + (\log n) \times n
\end{aligned}$$

Solving recurrence for Merge Sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned}
T(n) &= 2 \times T\left(\frac{n}{2}\right) + n \\
&= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + \frac{n}{2}) + n = 2^2 \times T\left(\frac{n}{2^2}\right) + 2n \\
&= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}) + 2n = 2^3 \times T\left(\frac{n}{2^3}\right) + 3n \\
&\vdots \\
&= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (\log n) \times n \\
&= 2^{\log n} \times T(1) + (\log n) \times n \\
&= n + n \log n = O(n \log n)
\end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$T(n) = 2 \times T\left(\frac{n}{2}\right) + 1$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\ &= 2 \times \left(2 \times T\left(\frac{n}{2^2}\right) + 1\right) + 1 \end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\ &= 2 \times \left(2 \times T\left(\frac{n}{2^2}\right) + 1\right) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + 1) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \\ &= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + 1) + (2^2 - 1) \end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned} T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\ &= 2 \times (2 \times T\left(\frac{n}{2^2}\right) + 1) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \\ &= 2^2 \times (2 \times T\left(\frac{n}{2^3}\right) + 1) + (2^2 - 1) = 2^3 \times T\left(\frac{n}{2^3}\right) + (2^3 - 1) \\ &\vdots \end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned}
T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\
&= 2 \times \left(2 \times T\left(\frac{n}{2^2}\right) + 1\right) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \\
&= 2^2 \times \left(2 \times T\left(\frac{n}{2^3}\right) + 1\right) + (2^2 - 1) = 2^3 \times T\left(\frac{n}{2^3}\right) + (2^3 - 1) \\
&\vdots \\
&= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (2^{\log n} - 1)
\end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned}
T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\
&= 2 \times \left(2 \times T\left(\frac{n}{2^2}\right) + 1\right) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \\
&= 2^2 \times \left(2 \times T\left(\frac{n}{2^3}\right) + 1\right) + (2^2 - 1) = 2^3 \times T\left(\frac{n}{2^3}\right) + (2^3 - 1) \\
&\vdots \\
&= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (2^{\log n} - 1) \\
&= n \times T(1) + (n - 1)
\end{aligned}$$

Solving Recurrence for Finding Sum/Max/Min

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \times T\left(\frac{n}{2}\right) + 1 & \text{if } n > 1 \end{cases}$$

Solving the recurrence

$$\begin{aligned}
T(n) &= 2 \times T\left(\frac{n}{2}\right) + 1 \\
&= 2 \times \left(2 \times T\left(\frac{n}{2^2}\right) + 1\right) + 1 = 2^2 \times T\left(\frac{n}{2^2}\right) + (2^2 - 1) \\
&= 2^2 \times \left(2 \times T\left(\frac{n}{2^3}\right) + 1\right) + (2^2 - 1) = 2^3 \times T\left(\frac{n}{2^3}\right) + (2^3 - 1) \\
&\vdots \\
&= 2^{\log n} \times T\left(\frac{n}{2^{\log n}}\right) + (2^{\log n} - 1) \\
&= n \times T(1) + (n - 1) \\
&= 2n - 1 = O(n)
\end{aligned}$$

Summary of recurrence and order of growth

$T(n) = 2 \times T(\frac{n}{2}) + 1$	$T(n)$ is $O(n)$
$T(n) = 2 \times T(\frac{n}{2}) + n$	$T(n)$ is $O(n \log n)$
$T(n) = 2 \times T(n-1) + 1$	$T(n)$ is $O(2^n)$

$T(n) = T(\frac{n}{2}) + 1 \Rightarrow T(n) = O(\log n)$

Summary of recurrence and order of growth

$T(n) = 2 \times T(\frac{n}{2}) + 1$ $\quad$ $T(n)$ is $O(n)$

$T(n) = 2 \times T(\frac{n}{2}) + n$ $\quad$ $T(n)$ is $O(n \log n)$

$T(n) = 2 \times T(n-1) + 1$ $\quad$ $T(n)$ is $O(2^n)$

$T(n) = T(\frac{n}{2}) + 1$ $\quad$ $T(n)$ is $O(\log n)$

What about Triomino Puzzle

What about Triomino Puzzle

- One instance of size 2^n is divided into four instances of size 2^{n-1}.

$$T(2^n) = \begin{cases} 1 & \text{if } n = 1 \\ 4 \times T(2^{n-1}) + 1 & \text{if } n > 1 \end{cases}$$

Summary

Summary: Fibonacci Numbers

Next: Dynamic Programming Algorithms

For note taking